PERSONNEL SYSTEMS / CONSCRIPTS PoC

Backend implementation and change report

Prepared for the system owner | 14 October 2026

Direct answer: I implemented the conscripts backend using your Markdown specification. I changed the earlier application design where it conflicted with that specification. I also made several explicit implementation choices and schema adaptations. This report identifies those differences and explains the replacement in each case.

Current result	Verified state
Local system	Running locally: frontend, API, SQL Server, Redis and background worker.
Application	http://localhost:5173
API health and documentation	http://localhost:8002/api/v1/health http://localhost:8002/docs
Backend / frontend checks	141 backend tests passed; 13 frontend tests passed; both TypeScript projects passed; pip check found no broken requirements.
User interface coverage	Existing login, administration and audit UI runs. Dedicated conscript import, completion, search and label screens remain placeholders. The new workflow is available through the API.

Rules retained from your MD

The four batches, official governorate codes, national-ID checks, approval/cancellation workflow, police-number-plus-year uniqueness, qualification durations, locking, Redis sessions, RBAC, SQL Server, RQ and SSE remain the core design.

Your original requirements were preserved

The supplied MD and PDF were read, not rewritten. The PDF carries the same requirements; its wide sample table is clipped in the export. The Markdown is treated as the governing version.

How to read this report: pages 2-3 explain delivered behavior and corrections to the old design; pages 4-5 list differences and additions; page 6 names remaining gaps; pages 7-8 document setup, evidence and operation.

02 / What the backend now does

The following is API/backend coverage. A working endpoint does not imply that its final user-facing screen has been implemented.

MD requirement	Delivered behavior
FR-1 to FR-4	Uploads XLSX, CSV, XLS and ACCDB; captures batch and year; validates national-ID structure, birth date, gender, official governorate and duplicates.
FR-5 / FR-6	Staged valid and invalid groups with per-row reasons. Administrator approval inserts valid rows atomically; cancellation inserts none. Database uniqueness is checked again at approval.
FR-7 to FR-9	Asynchronous rejected-row export to genuine XLSX or ACCDB, including source values and rejection reasons; protected download endpoints.
FR-10 to FR-14	Administrator audit access, timestamps, actor identity, before/after values, text search and date/time filters. Existing audit screen remains available; global text search is API-only.
FR-15 to FR-18	National-ID lookup, police-number entry, police-number/year uniqueness, suggested dates and explicit date submission. A start-date edit never silently rewrites the end date.
FR-19 / FR-20	Exact service durations: type 1 = 14 months; type 3 = 20; type 2 = 26; type 4 = 38. Missing or inconsistent dates are rejected.
FR-21: locking	Successful completion atomically locks the profile. Further changes require administrator permission. Revision checks prevent stale updates.
FR-21 to FR-24: retrieval/labels	Search by national ID, barcode, normalized name, or police number plus year. Printable Code 128 label includes supplied full name, police number and enlistment year.
FR-25 + processing diagrams	RQ background execution, Redis Pub/Sub, SSE snapshots, SQL counters, 250-row checkpoints and bounded retry/recovery. Parsing-stage granularity is explained on page 6.
FR-26 to FR-30	Redis-backed secure-cookie sessions, fixed administrator/operator roles, administrator account lifecycle, logout, expiry and session invalidation.
ERD and database constraints	New import/staging/error/recruit/export/document structures, foreign keys, unique indexes, SQL rowversion and validation constraints. Schema adaptations are listed on page 4.

The MD repeats FR-3 and FR-21. References above use their descriptive meaning so neither duplicated item is omitted.

03 / Corrections to the earlier system

These changes bring the old code into line with your new MD. They are not departures from your new requirements.

1. SQL session rows replaced by Redis sessions

Before: cookie sessions were stored in the SQL user_session table. **Now:** Redis stores session state under hashed cookie-token keys, with idle and absolute expiry. **Why:** FR-26 explicitly requires Redis. Old SQL session rows no longer authenticate users.

2. Direct permission assignment replaced by fixed roles

Before: administrators assigned arbitrary per-user permissions and copied editable permission templates. **Now:** each account has a system_administrator or data_entry_operator role; permission constants determine access. **Why:** FR-27 to FR-29 require RBAC and administrator-controlled role assignment.

3. Permission-template UI and API retired

The user-management screen now selects a role and displays derived permissions. Direct grant changes are rejected; template routes and menu entries were removed. Historical tables remain for compatibility, but they no longer grant access.

4. Skeleton-only module behavior replaced by functioning APIs

The old plan deliberately left conscript workflows unimplemented. The new recruits module implements upload, processing, review, approval, export, completion, search and labels. The earlier skeleton-only instructions were marked as superseded after your explicit request to build the backend.

5. Validation and dates follow the supplied rules

A separate official governorate reference was added from the MD, keeping old lookup data intact. Qualification IDs use the precise 14/20/26/38-month mapping. Both dates must be explicitly supplied; SQL constraints and a trigger reinforce important rules.

6. Background work and progress became operational

The setup now runs Redis and a separate RQ worker alongside the API. SQL-backed recovery and progress checkpoints replace synchronous or placeholder processing. Exports are generated by workers and downloaded through authenticated endpoints.

7. Documentation and generated client types updated

README, project instructions, permissions, dependency licences and setup scripts were updated. A backend guide and regenerated OpenAPI TypeScript definitions describe the actual endpoints. Historical documents are clearly identified.

Additional fix discovered during startup

A real job exposed POSIX process calls and path quoting problems in RQ 2.12 SpawnWorker on Windows. Added a Windows subprocess worker adapter and internal job_child entry point. RQ, Redis and SQL checkpoints remain unchanged; jobs still run in isolated child processes with timeout supervision. A regression test now exercises real import and export child processes.

04 / Intentional differences from the MD

The business workflow follows the MD, but the physical schema and queue delivery are not a literal copy of every diagram.

MD / diagram	What was used instead, and why
UUID user IDs	Existing app_user IDs remain integers, including related actor/uploader/completer foreign keys. New business records and jobs use UUIDs. This preserves the established core and its references. A full UUID user migration was not performed.
Separate parsed_* staging columns	Most parsed values are stored in parsed_data_json; normalized national ID and parsed_type_id remain explicit columns, with the qualification foreign key. This keeps normalized staging data together, but it changes the draft ERD shape.
Integer source_record_id	Stored as nullable text up to 100 characters. This retains source identifiers with leading zeroes or nonnumeric content; it does not reinterpret them as database identities.
New USERS / AUDIT_LOGS structures	Existing app_user and audit_log are extended/reused. Audit entity_id remains a string so it can reference integer core objects and UUID business objects. National ID/entity references are searchable in recorded before/after JSON rather than a new dedicated entity_reference column.
Plural table names and mixed text types	Uses repository-style singular snake_case names and generally NVARCHAR for text. Batch is INT with a 1-4 check rather than TINYINT. New business timestamps use DATETIME2(3). These are storage conventions, not changed batch/date rules.
Enqueue acknowledged before HTTP 202	The API commits a QUEUED SQL job and returns 202. A dispatcher delivers it to RQ, normally within 30 seconds. This prevents an accepted upload being lost if enqueue fails, but introduces a possible dispatch delay.
Native rowversion in the ERD	Native SQL rowversion is retained, with an additional integer revision exposed to clients. Administrator updates must supply the current revision; row locks and unique constraints protect concurrent writes.

Legacy conscript, import, template and session tables were preserved. The new workflow writes to recruit and the new staging tables. No automatic transfer of legacy conscript rows was implemented.

05 / Clarifications and added behavior

Decision / addition	Reason and effect
Complete role permissions	The short role table omits profile:insert and account-management codes even though FR-16/18/28/29 require them. Both roles can complete profiles; only administrators manage users. Existing administrator lookup maintenance is retained.
Within-file duplicate policy	The MD mandates database deduplication but does not choose a policy for repeated IDs within one file. Default: reject all occurrences. Optional keep_first mode retains the first. Policy is captured per import.
Male-only validation	The implementation rejects an even gender digit, consistent with the earlier conscript rules. The new MD defines gender encoding but does not separately state a permitted gender. Its sample has an even digit, so that sample is rejected. This is an interpretation requiring owner confirmation if women should be eligible.
Bounds and input conventions	Added 50 MiB upload / 100,000-row defaults, enlistment years 2000-2100, field length limits, UTF-8 CSV, required named headers and a selected first worksheet. These constraints make parsing predictable; the MD does not specify these bounds.
Barcode format and label dimensions	Retained the existing 12-digit year + batch + sequence barcode; prints Code 128 on 50 x 25 mm. The MD requires a scannable label and its contents, but does not specify the encoding or physical dimensions.
Additional legacy format	MDB input remains accepted as well as the four required formats, because the original sample is MDB. It does not replace ACCDB support.
Files and document handling	Private randomized storage names, SHA-256 checks and protected download paths were added. Document APIs implement the ERD relationship; current uploads accept PDF/PNG/JPG/JPEG and download as attachments.
Recovery and session safeguards	Added job heartbeats, a SQL dispatch record, application locks, bounded pre-start failures, and SQL account session versions. These support reliable recovery and immediate account revocation.
Safe preservation during export	Formula-like Excel source strings stay literal. A source rejection_reasons field is renamed to an unused source-prefixed name before adding the actual rejection column. Overlong Excel cells fail explicitly instead of silently truncating data.

No minimum recruit age or national-ID checksum algorithm was invented. End-of-month arithmetic uses calendar clamping; changing dates must still satisfy the required qualification duration.

06 / Remaining gaps and practical limits

The backend is implemented; the entire MD-defined application is not yet complete at the screen level. These limits are stated explicitly so the report does not overclaim delivery.

Conscript workflow screens are still placeholders

Upload/review, live progress, completion, unified search and label navigation do not yet have their final React screens. Their APIs are implemented. The current application provides the existing login and administration shell; API documentation can be used to exercise the backend.

Audit global search is not yet wired into the screen

GET /audit supports the q parameter and date/time ranges. The existing audit page exposes its previous filters, but no global q search box. Its dates are day-oriented, while the API accepts timestamps. Some newer action/entity names may appear as code labels until UI translations are added.

Parsing progress is coarser than a literal reading of FR-25

The worker publishes PARSING status, reads the bounded file, then knows total_rows. Processed/valid/invalid counters advance during 250-row validation commits. There is no row-by-row parsing progress before the initial read completes. On restart the source is reparsed, then validation resumes from its SQL checkpoint.

Administrator correction has a defined field scope

The current PATCH accepts name, police number and both dates with a required revision. It does not expose corrections to imported national ID, birth date, governorate, station, qualification, batch or year. Those broader correction rules were not specified in detail and remain to be designed if required.

Names and source data need business review

The label prints the supplied full name; validation does not enforce exactly four whitespace-separated tokens, because compound names make that unreliable. Governorate and qualification codes govern validation; the source display names are not independently reconciled to every reference name. Access source IDs should be unique for stable ordering.

A duplicate conflict at approval requires a new review cycle

Approval returns 409 and leaves the staged report intact if a national ID became occupied after validation. There is no separate revalidation endpoint; the administrator must cancel and upload again to generate a fresh report.

Deployment and large-volume behavior are not certified

The setup is for this local development PC. Production still needs supervised services, HTTPS, backup/retention policies, access-controlled shared storage and an offline deployment package. Parsing is bounded but in memory. No load test or physical label-printer/scanner acceptance test was performed.

Access processing depends on the Microsoft engine on a Windows worker. XLS uses xlrd; the automated evidence includes CSV/XLSX and native ACCDB, but not a dedicated XLS end-to-end fixture.

07 / Setup completed and verification

Component	Action / result
ODBC Driver 18	Installed Microsoft-signed ODBC Driver 18.7.1.1. Normal encrypted application connection verified.
Python environment	Installed Python 3.12.10 for this Windows user; repaired backend/.venv to use this PC's interpreter instead of the missing Lenovo path.
Application dependencies	Ran scripts/setup.ps1 successfully. Installed pinned backend requirements, checked frontend dependencies and added missing configuration keys without exposing passwords.
SQL Server and Redis	SQL Server 2022 and Redis 7.2 containers healthy; persistent volumes retained. Access 64-bit driver was already installed.
Database migration	Applied scripts 001-009 to the new local personnel database and verified safe reapplication. Tests use a separate personnel_test database.
API / frontend / worker	API on port 8002; frontend on 5173; corrected Windows RQ worker listening on the personnel queue. Logs and process IDs are in storage/logs.
Verification	141 backend tests passed; 13 frontend tests passed; both TypeScript projects passed; pip check found no broken requirements.

What was checked

HTTP 200 from the frontend and its API proxy; API health reports SQL and Redis reachable. The normal test script passed using ODBC 18. Native Access Arabic export/readback passed. Real Windows child processes completed import and export jobs. Administrator sign-in and required password change were checked.

Authentication and first sign-in

The owner explicitly approved the first administrator account. Username: admin. The generated temporary password is saved in storage/setup/admin-credentials.txt, excluded from Git. First sign-in requires a password change. Credentials are deliberately excluded from this PDF.

Local security settings

The application uses ODBC 18 with database encryption enabled. The development SQL container uses the existing TrustServerCertificate setting for its local certificate. Local HTTP uses SESSION_COOKIE_SECURE=false; production requires HTTPS and Secure cookies. No legacy-driver adapter is needed for the current setup.

Installer provenance

The Python installer was downloaded from python.org and its Python Software Foundation signature verified. The ODBC installer has a valid Microsoft signature. Access engine, Node.js and Docker Desktop were already present. Application package versions are pinned in the repository.

08 / Run instructions and audit trail

Open the system

http://localhost:5173
API health: http://localhost:8002/api/v1/health
API documentation: http://localhost:8002/docs

Start again after closing or restarting Windows

Open PowerShell in D:\Mirando\new work and run:

```
powershell -ExecutionPolicy Bypass -File scripts\dev.ps1
```

The script starts containers, applies idempotent database scripts and launches background application processes. Docker Desktop must be running.

Logs and stopping

Application logs and process IDs are in storage/logs. The startup script prints exact taskkill commands for the launched process trees. Stop database services with:

```
docker compose -f docker-compose.dev.yml stop
```

Persistent volumes remain. Do not remove volumes when you intend to retain data.

Location	Purpose
backend/app/recruits/	Import/export files, RQ jobs, validation, profile workflows, search, labels and attachments.
backend/app/core/ + admin/	Redis sessions, role-derived authorization, user lifecycle and audit search.
backend/sql/009_poc_backend.sql	Additive/idempotent PoC migration, references, constraints, indexes and trigger.
scripts/ + docker-compose.dev.yml	Setup, database initialization, application/worker startup and local services.
docs/backend-poc.md	API guide, rules, migration choices and operating instructions.
backend/tests/ + unit_tests/	Regression, rule and workflow checks with synthetic test data.

Requirements source: PoC 3f98b04e77e080368063da1908126936.md; accompanying 12-page PoC PDF. MD SHA-256: 591b369a7684400bf1f1cd24815f8fcc8959a5a1f5b6881a5d56ba0b10c34f52

Official setup references: Python 3.12.10 release; Microsoft ODBC Driver downloads. Implementation findings were checked against the local source code, migrations, tests and running services on the report date.